Module 4: Data Persistence and Management

SQL Essentials

SQL Joins Basics

INNER JOIN: Returns records that have matching values in both tables.

LEFT JOIN: Returns all records from the left table, and the matched records from the right table. If there is no match, the result is NULL on the right side.

RIGHT JOIN: Returns all records from the right table, and the matched records from the left table. If there is no match, the result is NULL on the left side.

FULL OUTER JOIN: Returns all records when there is a match in either left or right table. If there is no match, the result is NULL on the side that does not have a match.

CROSS JOIN: Returns the Cartesian product of the two tables, meaning it returns all possible combinations of records from both tables.

SELF JOIN: A table is joined with itself to compare rows within the same table.

```
-- INNER JOIN (only matching records)
SELECT
    o.order_id,
    c.first_name,
    c.last_name,
    o.order_date,
    o.total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_date >= '2024-01-01';

-- LEFT JOIN (all from left table)
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    COUNT(o.order_id) AS order_count,
    COALESCE(SUM(o.total_amount), 0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
```

```
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_spent DESC;

-- RIGHT JOIN (all from right table)
SELECT
    p.product_name,
    oi.quantity,
    oi.unit_price
FROM order_items oi
RIGHT JOIN products p ON oi.product_id = p.product_id;

-- CROSS JOIN (Cartesian product)
SELECT p1.product_name AS product1, p2.product_name AS product2
FROM products p1
CROSS JOIN products p2
WHERE p1.product_id < p2.product_id;

-- SELF JOIN
SELECT
    e1.employee_name AS employee,
    e2.employee_name AS manager
FROM employees e1
LEFT JOIN employees e2 ON e1.manager_id = e2.employee_id;

-- Multiple table joins
SELECT
    o.order_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    p.product_name,
    oi.quantity,
    oi.unit_price,
    (oi.quantity * oi.unit_price) AS subtotal
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
INNER JOIN order_items oi ON o.order_id = oi.order_id
INNER JOIN products p ON oi.product_id = p.product_id
WHERE o.status = 'DELIVERED'
ORDER BY o.order_date DESC;
```

SQL Transactions basics

An SQL transaction is a sequence of one or more SQL operations that are executed as a single unit of work. Transactions ensure data integrity and consistency, especially in multi-user environments.

Dirty reads, non-repeatable reads, and phantom reads are phenomena that can occur in concurrent transactions when isolation levels are not properly set.

Dirty read: A transaction reads data that has been modified by another transaction but not yet committed. If the other transaction rolls back, the data read is invalid. *Non-repeatable read:* A transaction reads the same row twice and gets different data each time because another transaction has modified and committed the data in between the two reads. *Phantom read:* A transaction reads a set of rows that satisfy a condition, but another transaction inserts or deletes rows that would have satisfied the condition, causing the first transaction to see a different set of rows if it re-executes the same query.

The above issues are called as *"read phenomena"* and can be mitigated by setting appropriate isolation levels for transactions. The standard isolation levels are:

- READ UNCOMMITTED: Allows dirty reads, non-repeatable reads, and phantom reads.
- READ COMMITTED: Prevents dirty reads but allows non-repeatable reads and phantom reads.
- REPEATABLE READ: Prevents dirty reads and non-repeatable reads but allows phantom reads.
- SERIALIZABLE: Prevents dirty reads, non-repeatable reads, and phantom reads by ensuring that transactions are executed in a completely isolated manner.

The key properties of transactions are often summarized by the acronym ACID:

- *Atomicity:* All operations within a transaction are treated as a single unit. Either all operations succeed, or none of them do. If any operation fails, the entire transaction is rolled back to maintain data integrity.
- *Consistency:* A transaction must transition the database from one valid state to another, ensuring that all data integrity constraints are maintained.
- *Isolation:* Transactions are isolated from each other, meaning that the intermediate state of a transaction is not visible to other transactions until it is committed. This prevents issues like dirty reads, non-repeatable reads, and phantom reads.
- *Durability:* Once a transaction is committed, its changes are permanent and will survive any subsequent system failures.

How to work with transactions in SQL?

1. START TRANSACTION: Begins a new transaction.
2. COMMIT: Saves all changes made during the transaction to the database.
3. ROLLBACK: Undoes all changes made during the transaction, reverting the database to its previous state.
4. SAVEPOINT: Creates a savepoint within a transaction, allowing you to roll back to that specific point without affecting the entire transaction. In such case you need to use ROLLBACK TO savepoint_name; to roll back to the savepoint.

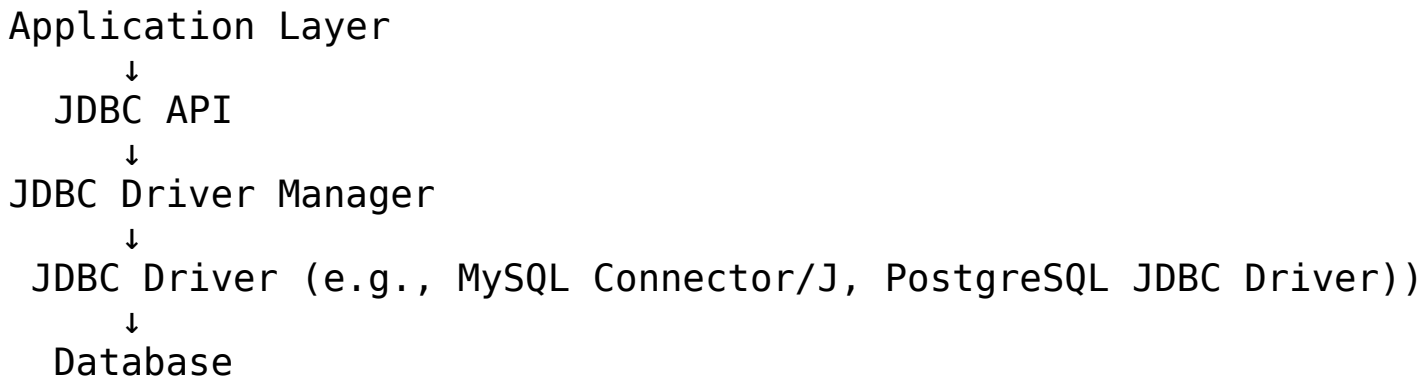
```
-- With out savepoints
-- Basic transaction
START TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
```

```
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
INSERT INTO transaction_log (from_account, to_account, amount) VALUES (1, 2, 100);

COMMIT;
-- or ROLLBACK; if error occurs

-- Transaction with savepoints
START TRANSACTION;

UPDATE inventory SET quantity = quantity - 5 WHERE product_id = 10;
SAVEPOINT after_inventory_update;

UPDATE orders SET status = 'CONFIRMED' WHERE order_id = 100;
SAVEPOINT after_order_update;

-- If needed, rollback to specific savepoint
-- ROLLBACK TO after_inventory_update;

COMMIT;
```

Introduction to JDBC

JDBC (Java Database Connectivity) is an API in Java that allows applications to connect to and interact with databases. It provides a standard set of interfaces for accessing relational databases, enabling developers to execute SQL queries, retrieve results, and manage database connections.

JDBC Architecture



JDBC Components:

- **DriverManager**: Manages driver connections
- **Connection**: Database connection
- **Statement**: Execute SQL
- **PreparedStatement**: Precompiled SQL with parameters (recommended)
- **CallableStatement**: Execute stored procedures

- **ResultSet**: Query results
- **SQLException**: Database errors

JDBC Basics

Below are the basic steps to use JDBC for database operations:

- Load the JDBC Driver
- Establish a Connection
- Create a Statement
- Process the ResultSet
- Close Resources

1. Load the JDBC Driver: The first step is to load the appropriate JDBC driver for the database you are connecting to. This is typically done using Class.forName(). Or else we can use DriverManager.registerDriver() to register the driver explicitly. However, with modern JDBC drivers, this step is often not required as they are automatically loaded when included in the classpath.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

2. Establish a Connection: Use the DriverManager.getConnection() method to establish a connection to the database.

```
String url = "jdbc:mysql://localhost:3306/mydatabase";
String username = "root";
String password = "password";
Connection connection = DriverManager.getConnection(url, username, password);
```

JDBC URL format: jdbc:subprotocol:subname Here jdbc is the protocol, subprotocol is the database type (e.g., mysql, postgresql), and subname includes the database location and name.

3. Create a Statement: Use the Connection object to create a Statement or PreparedStatement for executing SQL queries. If the executed statement is a SELECT query, it will return a ResultSet object containing the results. If it is an UPDATE, INSERT, or DELETE query, it will return an integer indicating the number of affected rows. Use the executeQuery method for SELECT statements and executeUpdate for INSERT, UPDATE, or DELETE statements. If you are executing a simple SQL query without parameters, you can use a Statement. For parameterized queries, use PreparedStatement.

```
Statement statement = connection.createStatement();
String sql = "SELECT * FROM customers";
ResultSet resultSet = statement.executeQuery(sql);
```

4. Process the ResultSet: Iterate through the ResultSet to retrieve data returned by the query. ResultSet provides various getter methods (e.g., getInt, getString) to access the data based on column names or indices. Initially the ResultSet cursor is positioned before the first row, so you need to call resultSet.next() to move it to the next row before accessing the data. The next() method returns false when there are no more rows to process.

```
while (resultSet.next()) {
    int id = resultSet.getInt("customer_id");
    String firstName = resultSet.getString("first_name");
    String lastName = resultSet.getString("last_name");
    System.out.println("ID: " + id + ", Name: " + firstName + " " + lastName);
}
```

5. Close Resources: Always close the ResultSet, Statement, and Connection to free up database resources.

```
resultSet.close();
statement.close();
connection.close();
```

PreparedStatement

PreparedStatement is a subclass of Statement that allows you to execute parameterized SQL queries. It provides better performance and security against SQL injection attacks.

```
String sql = "INSERT INTO customers (first_name, last_name) VALUES (?, ?)";
PreparedStatement preparedStatement = connection.prepareStatement(sql);
preparedStatement.setString(1, "John");
preparedStatement.setString(2, "Doe");
int rowsAffected = preparedStatement.executeUpdate();
System.out.println("Rows inserted: " + rowsAffected);
```

An SQL injection attack occurs when an attacker is able to manipulate the SQL query by injecting malicious input. Using PreparedStatement helps prevent this by treating input as data rather than executable code. For example, if a user input is directly concatenated into an SQL query, an attacker could input something like ”’; DROP TABLE customers; --” to delete the customers table. With PreparedStatement, the input is treated as a string literal, preventing such attacks.

With Statement (unsafe):

```
String userInput = "'; DROP TABLE customers; --";
```

```
String sql = "SELECT * FROM customers WHERE last_name = '" + userInput + "'";
Statement statement = connection.createStatement();
ResultSet resultSet = statement.executeQuery(sql);
```

With PreparedStatement (safe):

```
String userInput = "'; DROP TABLE customers; --";
String sql = "SELECT * FROM customers WHERE last_name = ?";
PreparedStatement preparedStatement = connection.prepareStatement(sql);
preparedStatement.setString(1, userInput);
ResultSet resultSet = preparedStatement.executeQuery();
```

Here the PreparedStatement treats the user input as a string literal, preventing the malicious input from being executed as part of the SQL query.

Transactions with JDBC

```
try {
     connection.setAutoCommit(false); // Start transaction

        // Execute multiple SQL statements

     connection.commit(); // Commit transaction
} catch (SQLException e) {
    connection.rollback(); // Rollback transaction on error
    e.printStackTrace();
} finally {
    connection.setAutoCommit(true); // Reset auto-commit mode
}
```

Connection Pooling

Connection pooling is a technique used to manage database connections efficiently. It allows applications to reuse existing connections rather than creating a new one for each request, which can improve performance and reduce resource consumption. A connection pool maintains a pool of active database connections that can be reused by multiple clients. When a client requests a connection, it is provided with an available connection from the pool. After the client is done with the connection, it is returned to the pool for reuse.

```
import javax.sql.DataSource;
import org.apache.commons.dbcp2.BasicDataSource;

// 1. Creating a connection pool using Apache Commons DBCP
```

```
BasicDataSource dataSource = new BasicDataSource();
dataSource.setUrl("jdbc:mysql://localhost:3306/mydatabase");
dataSource.setUsername("root");
dataSource.setPassword("password");
dataSource.setInitialSize(10); // Initial number of connections
dataSource.setMaxTotal(50); // Maximum number of connections

// 2. Getting a connection from the pool
Connection connection = dataSource.getConnection();

// 3. Using the connection to execute SQL queries
// etc...

// 4. Returning the connection to the pool
connection.close(); // This does not actually close the connection but returns it to the pool
```

Object-Relational Mapping (ORM) with JPA and Hibernate

Object-Relational Mapping (ORM) is a programming technique that allows developers to interact with a relational database using object-oriented programming languages. It abstracts the database interactions and provides a way to map database tables to Java classes, and rows to instances of those classes. This allows developers to work with data in a more intuitive and object-oriented way, without having to write complex SQL queries.

JPA (Java Persistence API)

JPA is a Java specification for ORM that provides a standard way to manage relational data in Java applications. It defines a set of interfaces and annotations for mapping Java objects to database tables and for performing CRUD (Create, Read, Update, Delete) operations. JPA is part of the Java EE platform and can be used in both Java SE and Java EE applications.

Features of JPA:

- Entity Mapping: JPA allows you to map Java classes to database tables using annotations or XML configuration. Each class that represents a database table is called an entity, and its fields represent the columns in the table.
- Entity Manager: JPA provides an EntityManager interface that is used to manage the lifecycle of entities, perform CRUD operations, and execute queries.
- Query Language: JPA includes a powerful query language called JPQL (Java Persistence Query Language) that allows you to write database queries using Java syntax instead of SQL.
- Transactions: JPA supports transactions, allowing you to group multiple operations into a single unit of work that can be committed or rolled back as needed.
- Caching: JPA provides a caching mechanism to improve performance by reducing the number of database hits.

Hibernate

Hibernate is a popular open-source ORM framework that implements the JPA specification. It provides additional features and capabilities beyond what is defined in JPA, making it a powerful tool for managing database interactions in Java applications. Hibernate is widely used in the industry and has a large community of developers.

Features of Hibernate:

- JPA Implementation: Hibernate is a JPA implementation, which means it fully supports the JPA specification and can be used as a drop-in replacement for any JPA provider.
- Advanced Mapping: Hibernate offers advanced mapping capabilities, such as support for inheritance, polymorphism, and complex associations between entities.
- Lazy Loading: Hibernate supports lazy loading, which allows you to load related entities on demand rather than loading them all at once, improving performance.
- Caching: Hibernate provides a powerful caching mechanism that includes both first-level (session) and second-level (shared) caches to optimize database access.
- Querying: In addition to JPQL, Hibernate supports native SQL queries and a criteria API for building queries programmatically.

```
// Example of a JPA entity class

@Entity
@Table(name = "customers")
public class Customer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "first_name")
    private String firstName;

    @Column(name = "last_name")
    private String lastName;

    // Getters and setters
}

// Example of using JPA EntityManager to perform CRUD operations

EntityManagerFactory emf = Persistence.createEntityManagerFactory("my-persistence-unit");
EntityManager em = emf.createEntityManager();

// Create a new customer
Customer customer = new Customer();
customer.setFirstName("John");
customer.setLastName("Doe");
```

```
em.getTransaction().begin();
em.persist(customer);
em.getTransaction().commit();

// Read a customer
Customer foundCustomer = em.find(Customer.class, customer.getId());

// Update a customer
em.getTransaction().begin();
foundCustomer.setLastName("Smith");
em.getTransaction().commit();

// Delete a customer
em.getTransaction().begin();
em.remove(foundCustomer);
em.getTransaction().commit();
```

In this example, we define a JPA entity class `Customer` that maps to a database table named “customers”. We use annotations to specify the mapping between the class and the table, as well as the mapping of fields to columns. We then use the `EntityManager` to perform CRUD operations on the `Customer` entity, demonstrating how to create, read, update, and delete records in the database using JPA.

Integration of JPA with SpringBoot

SpringBoot provides excellent support for integrating JPA into your applications. It simplifies the configuration and setup process, allowing you to quickly get started with JPA and Hibernate. To integrate JPA with Spring Boot, you typically need to include the necessary dependencies in your project, configure the database connection, and define your entity classes and repositories.

1. Add Dependencies: In your `pom.xml` (for Maven) or `build.gradle` (for Gradle), include the following dependencies:

```
<!-- Spring Boot Starter Data JPA -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<!-- Database Driver (e.g., MySQL) -->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
</dependency>
```

2. Configure Database Connection: In your `application.properties` or `application.yml`, configure the database connection properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/mydatabase
spring.datasource.username=root
spring.datasource.password=password
spring.jpa.hibernate.ddl-auto=update
```

3. Define Entity Classes: Create your JPA entity classes and annotate them with `@Entity` and other relevant annotations to map them to database tables.

```
@Entity
@Table(name = "customers")
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "first_name")
    private String firstName;

    @Column(name = "last_name")
    private String lastName;

    // Getters and setters

}
```

4. Create Repository Interfaces: Define repository interfaces that extend `JpaRepository` or `CrudRepository` to provide CRUD operations for your entities. Spring Data JPA will automatically generate the implementation of these interfaces at runtime, allowing you to perform database operations without writing any SQL or boilerplate code. Here is an example of a repository interface for the `Customer` entity. Here Long is the type of the primary key of the Customer entity.

```
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    // Custom query methods can be defined here
}
```

5. Use the Repository in Your Service or Controller: You can now inject the repository into your service or controller classes and use it to perform database operations. It will

automatically handle the implementation of the repository and the database interactions, allowing you to focus on your business logic.

```
@Service
public class CustomerService {

    @Autowired
    private CustomerRepository customerRepository;

    public List<Customer> getAllCustomers() {
        return customerRepository.findAll();
    }

    public Customer getCustomerById(Long id) {
        return customerRepository.findById(id).orElse(null);
    }

    public Customer createCustomer(Customer customer) {
        return customerRepository.save(customer);
    }

    public void deleteCustomer(Long id) {
        customerRepository.deleteById(id);
    }
}
```

In this example, we have defined a `Customer` entity class and a `CustomerRepository` interface that extends `JpaRepository`. We then use the repository in a service class to perform CRUD operations on the `Customer` entity. Spring Boot will automatically handle the implementation of the repository and the database interactions, allowing you to focus on your business logic.

Repository patterns using Spring Data JPA, Pagination, sorting, and custom queries with JPQL

Spring Data JPA is a part of the larger Spring Data family, which provides a consistent and easy-to-use approach to data access in Spring applications. It simplifies the implementation of data access layers by providing a set of interfaces and annotations that allow you to define repository interfaces for your entities. Spring Data JPA provides built-in support for pagination and sorting, allowing you to easily retrieve subsets of data and sort results based on specific criteria. It also allows you to define custom queries using JPQL (Java Persistence Query Language) or native SQL.

1. Pagination and Sorting: Spring Data JPA provides the `Pageable` interface for pagination and the `Sort` class for sorting. You can use these in your repository methods to retrieve paginated and sorted results.

```
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    // Method to find customers with pagination and sorting
    Page<Customer> findAll(Pageable pageable);
}
```

In your service or controller, you can create a `Pageable` object to specify the page number, page size, and sorting criteria:

```
Pageable pageable = PageRequest.of(0, 10, Sort.by("lastName").ascending());
Page<Customer> customersPage = customerRepository.findAll(pageable);
List<Customer> customers = customersPage.getContent();
```

2. Custom Queries with JPQL: You can define custom queries in your repository interface using the `@Query` annotation. This allows you to write JPQL queries to retrieve data based on specific criteria.

```
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    // Custom query to find customers by last name
    @Query("SELECT c FROM Customer c WHERE c.lastName = :lastName")
    List<Customer> findByLastName(@Param("lastName") String lastName);
}
```

In this example, we define a custom query to find customers by their last name. The `@Query` annotation allows us to write a JPQL query that selects customers based on the `lastName` parameter. We can then call this method in our service or controller to retrieve the desired results.

```
List<Customer> customers = customerRepository.findByLastName("Smith");
```

In summary, Spring Data JPA provides powerful features for managing data access in Spring applications. It simplifies the implementation of repositories, supports pagination and sorting, and allows for custom queries using JPQL. By leveraging these features, you can efficiently manage your data and build robust applications with ease.

General Naming Conventions for repository methods

Some of the important naming conventions for repository methods in Spring Data JPA include:

- findBy: Used to retrieve entities based on specific criteria (e.g., findByLastName, findByEmail).

- countBy: Used to count the number of entities that match specific criteria (e.g., countByStatus).
- deleteBy: Used to delete entities based on specific criteria (e.g., deleteById).
- existsBy: Used to check if an entity exists based on specific criteria (e.g., existsByUsername).
- findAll: Used to retrieve all entities (e.g., findAll).
- findById: Used to retrieve an entity by its primary key (e.g., findById).
- save: Used to save an entity (e.g., save).
- saveAll: Used to save a list of entities (e.g., saveAll).
- findBy[Property]And[Property]: Used to retrieve entities based on multiple criteria (e.g., findByFirstNameAndLastName).
- findBy[Property]Or[Property]: Used to retrieve entities based on either of multiple criteria (e.g., findByFirstNameOrLastName).
- findBy[Property]GreaterThan: Used to retrieve entities where a property is greater than a specified value (e.g., findByAgeGreaterThan).
- findBy[Property]LessThan: Used to retrieve entities where a property is less than a specified value (e.g., findByAgeLessThan).
- findBy[Property]Like: Used to retrieve entities where a property matches a pattern (e.g., findByFirstNameLike(“John%”)).

Best Practices for JPA and Hibernate

- Use `@Entity` to define your entity classes and map them to database tables.
- Use `@Id` to specify the primary key of your entity and `@GeneratedValue` to specify how the primary key should be generated.
- Use `@Column` to map entity fields to database columns and specify column properties.
- Use `@OneToMany`, `@ManyToOne`, `@OneToOne`, and `@ManyToMany` annotations to define relationships between entities.
- Use `@Transactional` to manage transactions in your service layer.
- Use `@Repository` to annotate your repository interfaces and enable exception translation.
- Use `@Service` to annotate your service classes and encapsulate business logic.
- Use `@Autowired` to inject dependencies into your classes.
- Use `@Query` to define custom queries when needed, and prefer JPQL over native SQL for better portability.
- Use pagination and sorting to efficiently retrieve large datasets.
- Avoid using `fetch = FetchType.EAGER` for relationships unless necessary, as it can lead to performance issues. Use `fetch = FetchType.LAZY` instead and fetch related

entities on demand.

- Use DTOs (Data Transfer Objects) to transfer data between layers and avoid exposing entity classes directly to the client.
- Handle exceptions properly and use Spring’s exception translation to convert database exceptions into more meaningful exceptions for your application.
- Use caching to improve performance, but be mindful of cache consistency and invalidation strategies.
- Regularly monitor and optimize your database queries to ensure good performance, especially as your application scales. Use tools like Hibernate’s SQL logging to analyze the generated SQL queries and identify potential performance bottlenecks.